

Code of the day ...

- What does the following code do?

```
int x, y;           // input values to compare signs
initialize x and y;
bool f = ((x ^ y) < 0); // what can you tell by “f”
```

ECE 759

High Performance Computing for Applications in Engineering

[Spring 2025]

Instruction-level Parallelism and Task-level Parallelism

2025/01/31

Before we get started...

- Quick overview, things discussed last time
 - Do not run code on the log-in node of Euler. CAE will suspend your account. Use Slurm to launch you exe.
 - Discuss about the concept of instruction. Your program is a bunch of them instructions
 - Discuss how transistors are organized to work together so that they end up executing them instructions
- Purpose of today's lecture:
 - Cover registers. Store information & there's no faster way to access info than from registers
 - Instruction-level parallelism (ILP)
 - Thread-level parallelism (TLP)
 - Judging how much time a program takes to complete
- Miscellaneous
 - Assignment #1 due 2/3 23:59 pm
 - If interested in taking this class but still in the waitlist, it will be offered next Fall/Spring

Registers

From High-level Source Code to Machine Instructions

- Line of C code:

```
a[4] = delta + a[3]; //line of C code
```

- MIPS assembly code generated by the compiler (three instructions):

```
lw  $t0, 12($s2) # reg $t0 gets value stored 12 bytes from address in reg $s2
add $t0, $s4, $t0 # reg $t0 gets the sum of values stored in reg $s4 and reg $t0
sw  $t0, 16($s2) # store what's in reg $t0 at mem location 16 bytes from address in reg $s2
```

- Set of three corresponding MIPS instructions produced by the compiler:

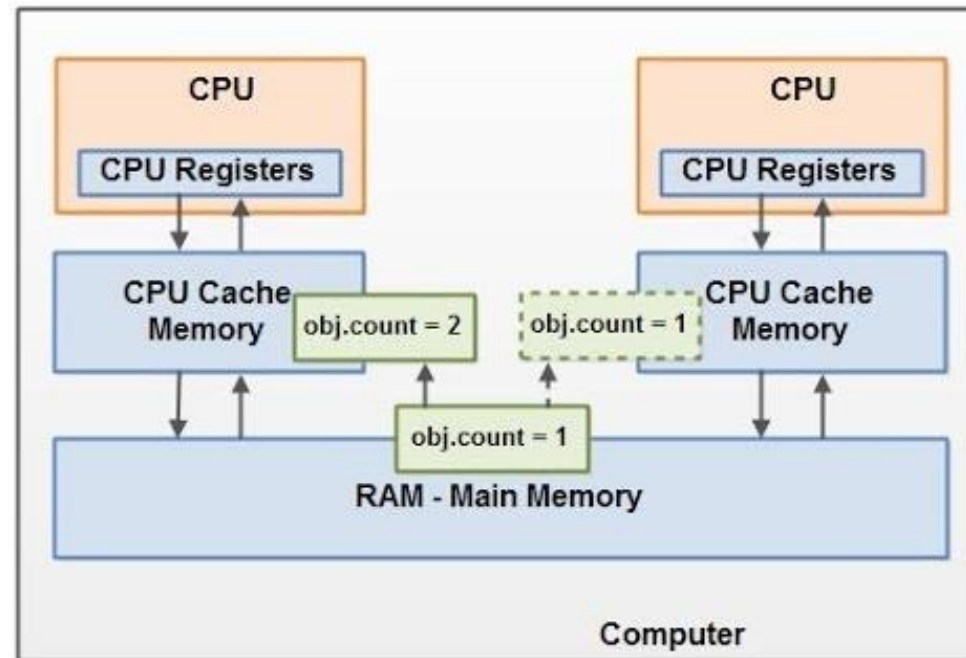
```
10001110010010000000000000001100
00000010100010000100000000100000
10101110010010000000000000001000
```

Fetch-Decode-Execute (FDX) Cycle

- In the fetch-decode-execute (FDX) cycle going through CU and ALU
 - CU – responsible for controlling the process that will decide what the input instruction needs to do
 - Type I (immediate data available), R (function), J (jump)
 - ALU – does the instructed work
 - Add/Sub/Mul
 - Special functional unit: sin, cos, etc.
 - ...

Where do we store instructions?

- The **instruction** that is being executed needs to be **stored somewhere**
- Executing an instruction usually involves getting input values and generating output values
 - This **data** needs to be **stored somewhere**

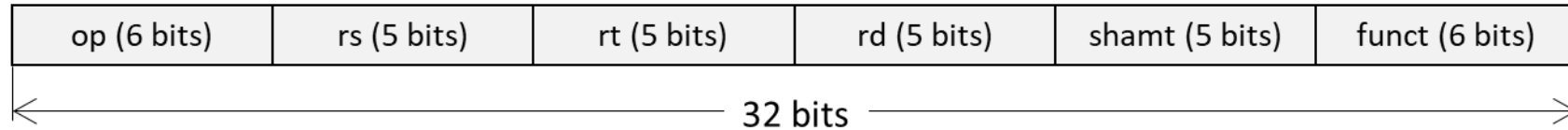


Everything is stored in “register” at the processor level

- A register is a hardware asset whose role is to store data
 - Variable value
 - Instruction
 - Address
 - ...
- A register is the storage type with shortest latency – part of the CU & ALU combo
 - Latency is as fast as hundreds of picoseconds
- When processing one instruction, the chip needs all instruction-related operands in registers

Registers are great! But we don't have many of them

- Some chips have close to 100 MB of L3 cache but only a few tens of registers
- Why? The **size** and **number** of registers are specific to an ISA



- The size of the register is typically limited to 32 bits or 64 bits
 - Only five bits used to encode the register ID. With five bits, you can reference 2^5 registers
- More registers → wider ISA length → higher hardware/software design cost

Register Types (1/3)

- **Current Instruction Register (CIR)** – holds the instruction that is executed
 - Sometimes known as “instruction register” IR
- **Program Counter (PC)** – holds address of the instruction to be executed next
 - NOTE: unlike CIR, PC contains *address* of the instruction, not actual instruction
 - Sometimes called IAR: Instruction Address Register
- **Memory Data Register (MDR)** – holds data read in from memory or produced by the ALU and waiting to be stored in memory
- **Memory Address Register (MAR)** – holds address of RAM memory location where input/output data is supposed to be read in/written out
 - NOTE: unlike MDR, MAR contains *address* of a mem location, not actual data
- **Return Address (RA)** – the address where upon finishing a sequence of instructions, the execution should jump and commence with the execution of subsequent instruction

Register Types (2/3)

- Registers on the previous slide are just a subset of most ISA
- There are several other registers commonly used by many chip designs
 - Registers for **Subroutine Arguments** (4) – a0 through a3
 - Registers for **Temporary Variables** (10) – t0 through t9
 - Registers for **Saved Temporary Variables** (8) – s0 through s7
 - Saved between function calls

Register Types (3/3)

- Several other registers are involved in handling function calls:
 - **Stack Pointer** (sp) – A register that keeps track of the top of the stack in memory
 - **Global Pointer** (gp) – A register that helps access global variables and constants stored in a specific 64KB memory block
 - **Frame Pointer** (fp) – A register that marks the start of a function's memory frame, usually pointing to the stack's previous position before the function was called

Final Thoughts on Registers

- Registers: we love them, we can't have enough of them because the limit of bit width
 - Ex: Reference registers using only 5 bits (i.e., 32 registers at maximum)
- Increasing their number is not straightforward
 - Need to change the very nature of ISA (vocabulary of hardware design)
 - Need to change the design and microarchitecture of the chip
- Examples:
 - In 32-bit MIPS ISA, there are 32 registers
 - GP100 GPU (Pascal), register file size: 14,336 MB (~3.5 million registers)
 - Seems impressive but it's shared among many thousands of threads
 - $56 \text{ SMs} * 2048 \text{ threads/SM} * 32 \text{ registers/thread} = 3.6 \text{ million registers needed}$ (almost exactly what the hardware made available...)

Pipelining

“Nothing is particularly hard if you divide it into small jobs.”

--Henry Ford, founder of Ford Motor Company [1863 –1947]

Why pipelining was embraced: Too many cooks spoil the broth

If you have all the cooks working on the same dish, you will spoil the broth very easily ...



Pipeline parallelism



Pipeline parallelism: Instead, you can partition the recipe into several dependent or independent steps and distribute these steps across cooks

Pipeline is everywhere especially in the manufacturing industry

- Historical change of the manufacturing assembly line. Manual assembly line (left); Hyundai assembly line in Chennai, India (right)

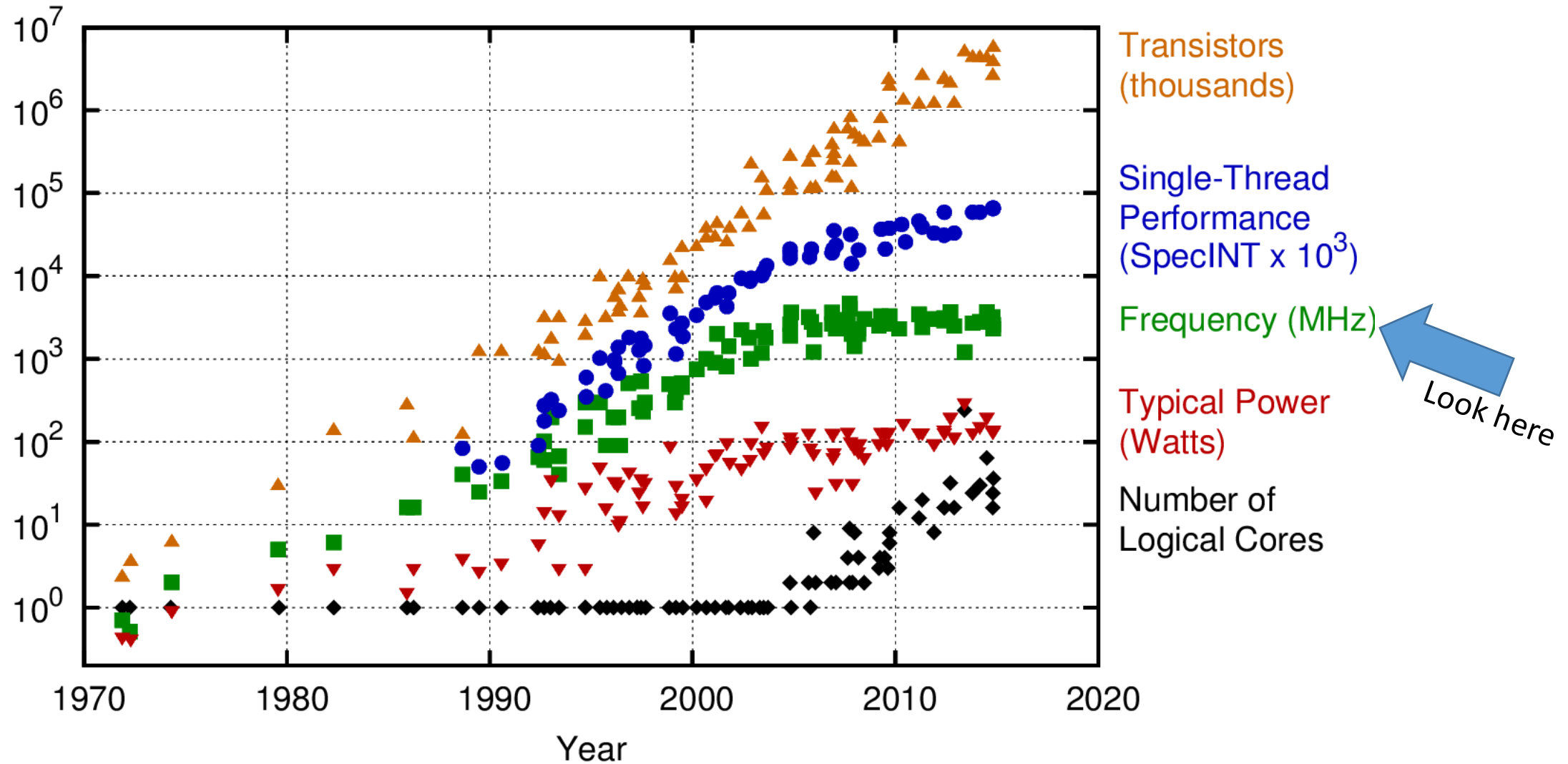


<https://www.rediff.com/business/report/hyundai-uses-580-robots-in-chennai-plant-maruti-5000/20180530.htm>

Clock Cycle

- Assembly line moves at a regular interval (i.e., the time between each movement)
 - Each stage must complete its job within a time limit so the product can be passed to the next stage
 - Once the last stage completes, a vehicle gets finished/manufactured
- Process applies the similar pipeline concept to parallelize the FDX cycle among instructions
 - Each stage corresponds to one step of processing an instruction
 - Ex: fetch, decode, execute, etc.
 - Movement from one stage to the next stage is controlled by “clock cycle”
- What is clock cycle?
 - Time between two consecutive ticks of the clock
 - Ex: A **1 GHz** processor ticks **1 billion (10^9) times per second**
 - Higher clock speed usually means faster performance

50 years of microprocessor trends: clock ticks at 4 GHz (give or take)



Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2015 by K. Rupp

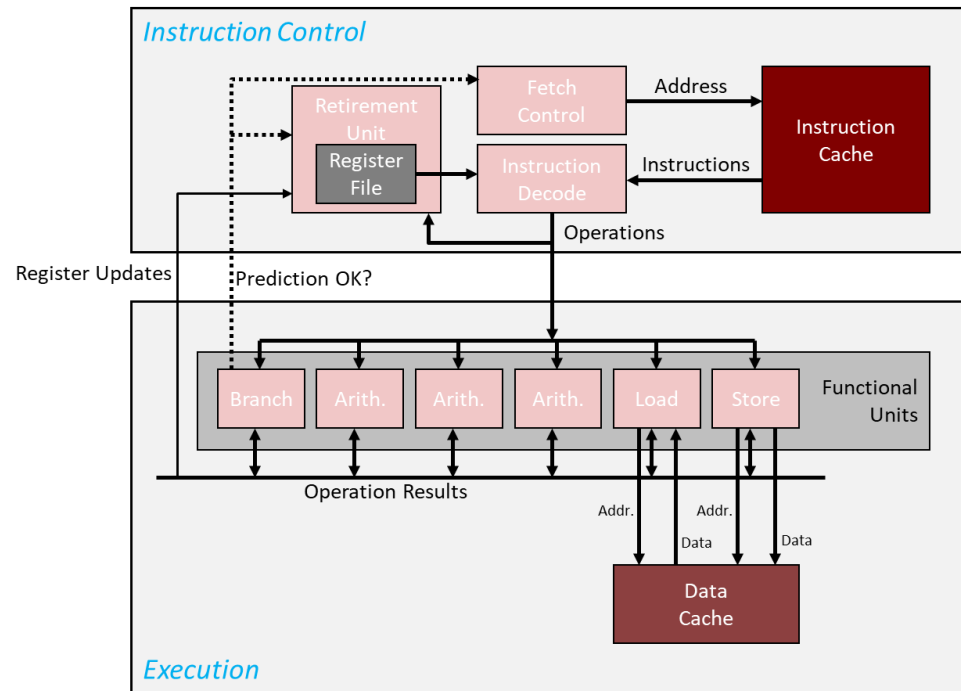
FDX cycle: sounds like 3 stages, but ends up being 5 or more

- FDX cycle: Fetch, Decode, Execute - carried out for **each** machine instruction, 3 stages
- Upon a closer look at what gets fetched (instructions and data) and then what happens upon execution leads to a generic five-stage process associated with an instruction
 - Stage 1: Fetch an instruction
 - Stage 2: Decode the instruction
 - Stage 3: Data access
 - Stage 4: Execute the operation (e.g., might be a request to calculate an address)
 - Stage 5: Write-back into register file
- “FDX cycle” has nothing to do with “cycle time”, which is the time between two clock ticks
 - Think about it like this: the recipe of a plate tells how to create the food step by step, and it has nothing to do with the time; each step may take several clock cycles to finish

In fact, today you have 15 stages or so in a pipeline

1. Instruction Fetch (IF)
2. Instruction Decode (ID)
3. Register Renaming
4. Dispatch
5. Execution Units (EX)
6. Memory Access (MEM)
7. Write-Back (WB)
8. Commit
9. Retirement
10. Exception Handling

Visualization of the FDX cycle on the Haswell Microarchitectures

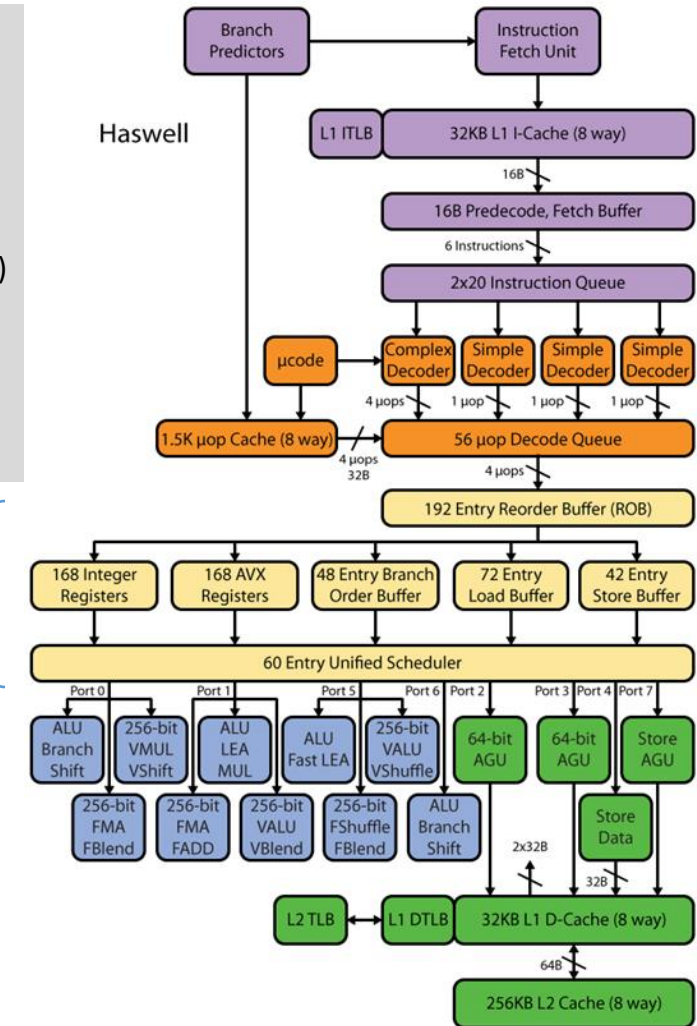


Microarchitecture components:

- Instruction pre-fetch support (purple)
- Instruction decoding support (orange)
 - CISC into uops
 - Can be regarded as CISC to RISC step
- Instruction Scheduling support (yellowish)
- Instruction execution
 - Arithmetic (blueish)
 - Memory related (green)

In this segment,
the focus is here...

... and here



Pipeline the execution of instructions

- Assume a five-stage pipeline
 - Keep in mind in modern architectures, it gets to be somewhere between 10-20 stages deep
- Key idea of pipeline is to overlap the execution of instructions across different stages
 - Ex: the following tasks can run simultaneously when processing five instructions, assuming they are linearly dependent on each other (1→2→3→4→5)
 - Instruction 1 is in the 5th stage of the FDX cycle
 - Instruction 2 is in the 4th stage of the FDX cycle
 - Instruction 3 is in the 3rd stage of the FDX cycle
 - Instruction 4 is in the 2nd stage of the FDX cycle
 - Instruction 5 is in the 1st stage of the FDX cycle

Rules of thumb: costs to eXecute an operation > 1 clock cycle

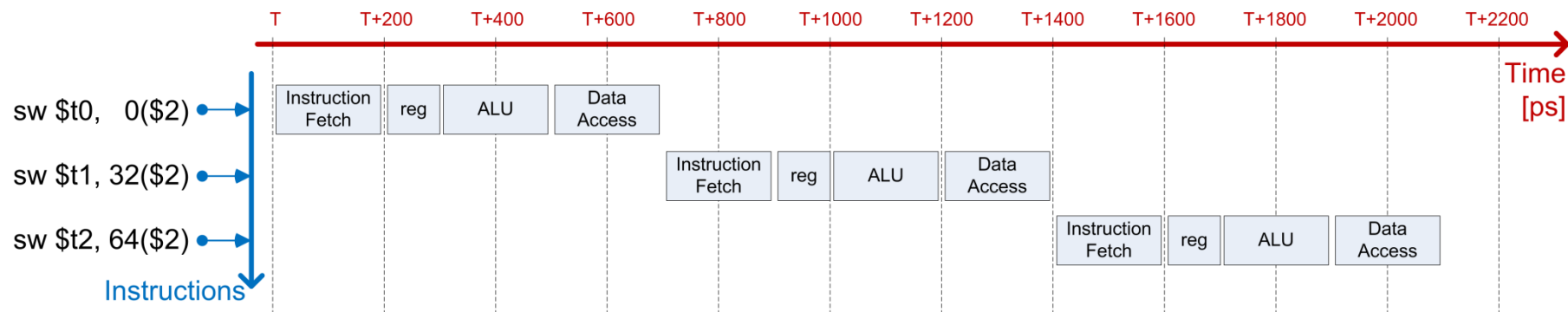
[refers to older MIPS ISA; cost of X in FD~~X~~]

- Four clock cycles for SW (store word)
 - Fetch $\rightarrow$ Decode $\rightarrow$ Execute $\rightarrow$ Memory Access
- Five clock cycles for LW (load word)
 - Instruction Fetch $\rightarrow$ Decode $\rightarrow$ Execute $\rightarrow$ Memory Access $\rightarrow$ Write-Back
- Five clock cycles for add
 - Instruction Fetch $\rightarrow$ Decode $\rightarrow$ Execute $\rightarrow$ Memory Access $\rightarrow$ Write-Back
- Etc.

Example: Sequential execution of three SW instructions

```
sw    $t0, 0($s2) //store what is in register $t0 at mem location 0 bytes from address in register $s2
sw    $t1, 32($s2) //store what is in register $t1 at mem location 32 bytes from address in register $s2
sw    $t2, 64($s2) //store what is in register $t0 at mem location 64 bytes from address in register $s2
```

- Case 1: No pipelining – 2100 picoseconds [ps]



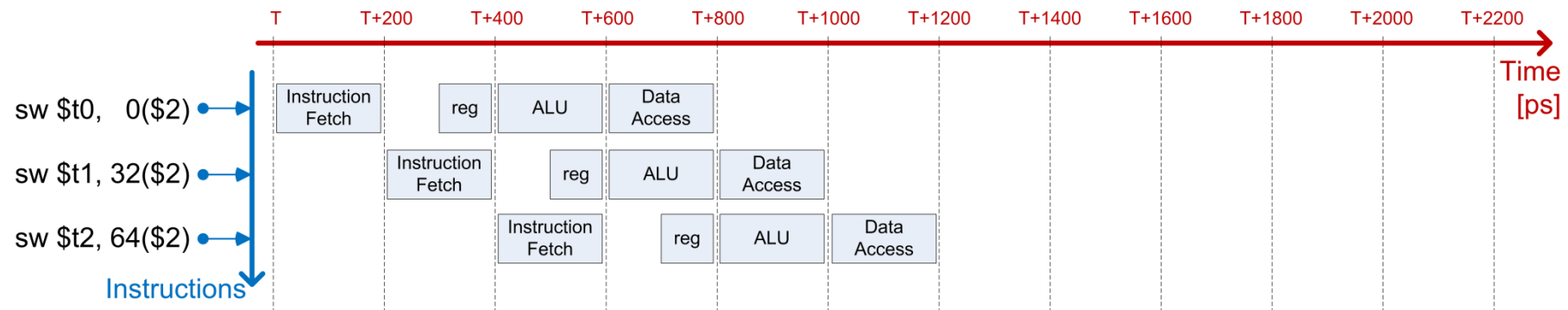
Example: Pipelined execution of three SW instructions

sw \$t0, 0(\$s2)

sw \$t1, 32(\$s2)

sw \$t2, 64(\$s2)

- Case 2: With pipelining – 1200 picoseconds [ps] (5 GHz clock cycle)



Load balance in Pipeline

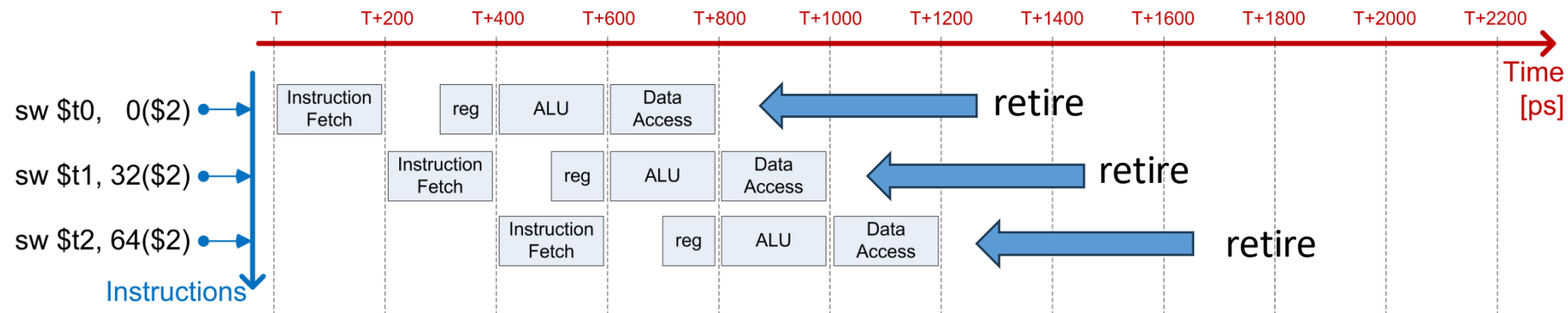
- An ideal situation is when each stage pipeline takes the same amount of time for completion
 - The pipeline is then **balanced**
- If pipeline not balanced, there will be stations waiting for the laggard station at every cycle
- A simple solution: If there is a station that takes a significantly longer time since it does significantly more than other stations, it should be broken into two and the length of the pipeline increases by one station
 - Leads to deeper pipelines

Pipelining Benefits: faster execution & no code rewrite

- **Benefit 1: faster execution**, due to simultaneously working on multiple instructions
 - Assume that you have
 1. A very large number of instructions
 2. Balanced stages
 3. A pipeline that is larger than or equal to the number “ p ” of stages associated with the typical ISA instruction
 - If 1 through 3 above hold, in a first order approximation, speed-up you get out of pipelining is approximately “ p ”
- **Benefit 2: no code rewrite** required on the user’s part
 - This kind of parallel processing of stages is transparent to the user (i.e., handled directly by hardware)
 - Unlike GPU or multicore parallel computing, you don’t have to do anything to benefit from it

Pipelining Benefits: instruction throughput

- Benefit 3: One instruction retired at each clock cycle
 - When the pipeline is busy – fully saturated and filled with instruction stages



- Pipelining does not decrease the time to complete one instruction but rather it increases the throughput of the processor by overlapping different stages of processing of different instructions

Pipelining Hazards

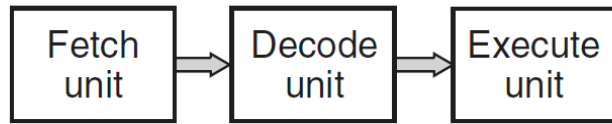
- Q: if deep pipelines are good, why not make them deeper?
- A: deep pipelines are plagued by **pipelining hazards**
 - **Structural hazards:** Happen when the CPU doesn't have enough hardware to handle multiple instructions at the same time. Think of it like too many cooks trying to use the same stove in a busy kitchen
 - **Data hazards:** Occur when one instruction needs the result of a previous instruction that hasn't finished yet. It's like waiting for an ingredient to be chopped before you can cook with it
 - **Control hazards:** Happen when the CPU guesses wrong on a branch (e.g., an if statement). If the guess is wrong, it has to throw away the incorrect work and start over, like taking the wrong turn and needing to backtrack

Pipelining vs. Multiple-Issue (1/3)

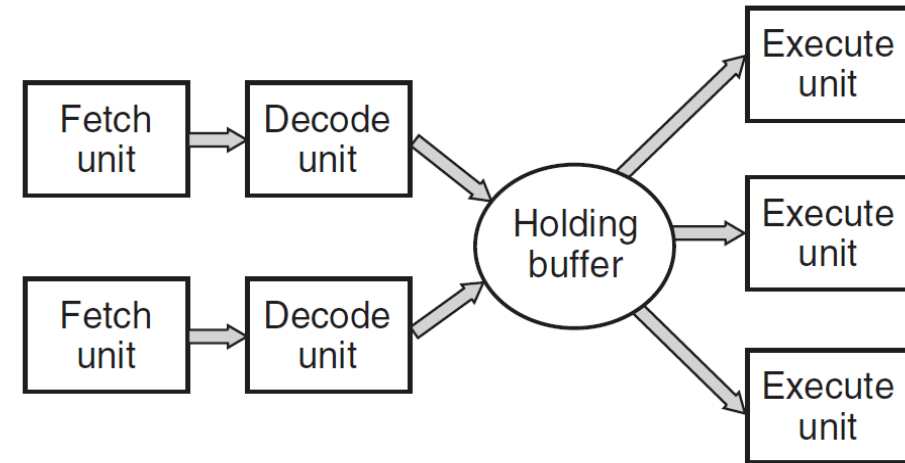
- What is multiple-use?
 - In sequential computing, a multiple-issue processor core has the hardware to issue more than one instruction per cycle
 - Analogy: A cook can simultaneously sprinkle white and black peppers on two lines with his/her two hands
- “Multiple-issue” is different from pipeline, which is another way of speeding up execution
- Example: the C code below leads to a set of instructions that can be launched for execution at the same time
 - Why?

```
int a, b;  
float c, d;  
//some code setting up a, b, c, d  
a += b;  
c += d;
```

Pipelining vs. Multiple-Issue (2/3)



(a) Three-stage pipeline



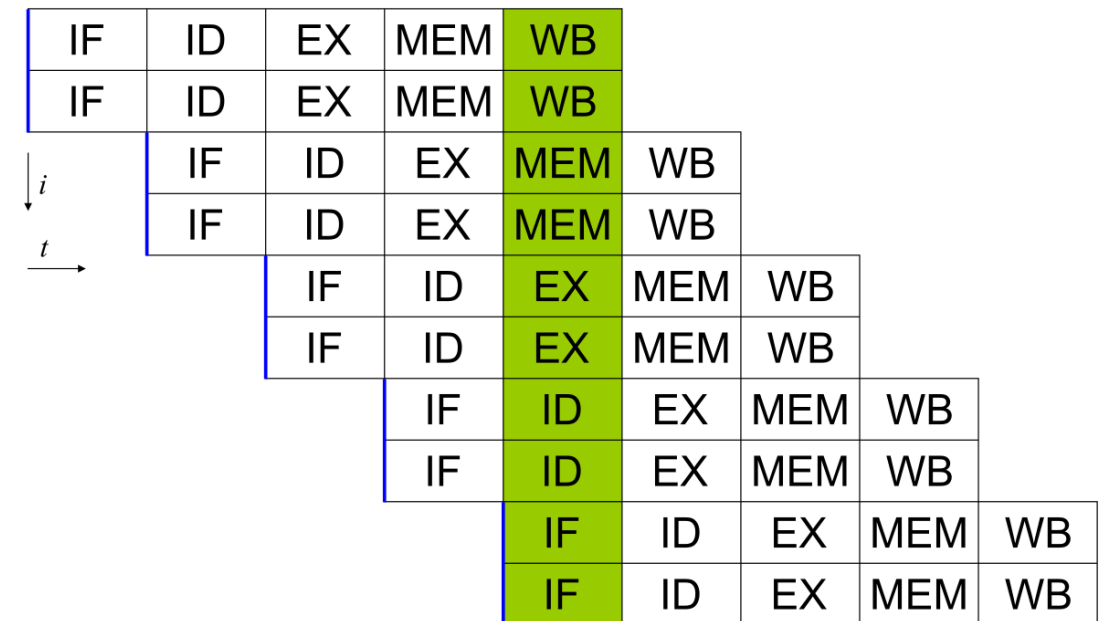
(b) Multiple issue processor

Pipelining vs. Multiple-Issue (3/3)

- In reality, more than one instruction is processed by the **same core** in the **same clock cycle**
- Multiple-Issue can be done statically or dynamically
 - **Static** multiple-issue:
 - Predefined, doesn't change at run time
 - Who uses it: Intel, in AVX512 (but not only)
 - **Dynamic** multiple-issue:
 - Determined at run time, the chip has dedicated hardware resources that can identify and execute additional work
 - Who uses it: Intel, AMD, IBM, NVIDIA, etc.

Dynamic Multiple-issue Processor

- CU supervises what instructions can be issued simultaneously by checking data dependencies
 - However, checking for dependencies is complex, requires high cost in time and energy
 - Also, we need to check if the result is consistent with what it is supposed to be
 - “ok”: no difference if the instructions are executed “multiple-issue” or “single issue”
- NOTE: a chip that can do multiple-issue is also called a **superscalar** architecture
 - A superscalar architecture can explore pipeline too
 - Ex: two-issue instruction pipeline (right figure)



Arriving at “Instruction-Level Parallelism” (ILP)

- A parallelism type explored at the instruction level, typically done directly by hardware
 - Pipeline, multiple-issue, out-of-order execution (OOOE), speculative execution, branch prediction, etc.
 - In most cases, you have very little control – which is good since hardware directly does this for you
 - Nevertheless, the way you write your code have a huge impact on how ILP can be maximally explored
 - Ex: temporal and spatial locality-aware for-loop and memory access
- ILP pursues various strategies to execute simultaneously multiple instructions
- Important: these instructions belong to the same program/process or one thread
 - “process” – how the operating systems calls your program
 - Ex: When you run “task6” in HW01, the operating system creates a process for it

Short summary of common ILP strategies

- **Instruction pipelining**: execution of multiple instructions can be partially overlapped; each instructions divided into sub-steps (termed: micro-operations)
- **Superscalar execution**: multiple execution units are used to execute multiple instructions in parallel
- **Out-of-order execution**: instructions execute in any order but without violating data dependencies
- **Register renaming**: a technique used to avoid data hazards and thus lead to unnecessary serialization of program instructions caused by the reuse of registers
- **Speculative execution**: allows the execution of complete instructions or parts of instructions before being sure whether this execution is required
- **Branch prediction**: used to avoid delays of control flow. Used in combination with speculative execution.

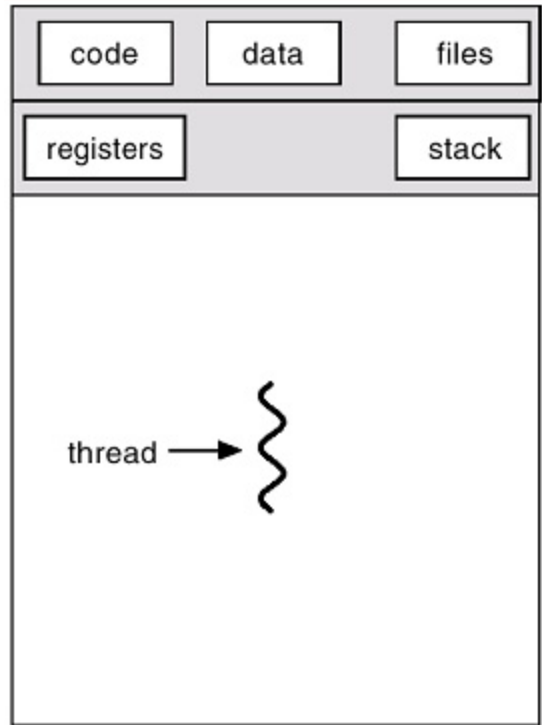
Thread Level Parallelism (TLP) [happens on a single core]

- A type of parallelism explored at the software level
 - You design a parallel algorithm (i.e., an algorithm that can run in parallel, such as parallel sort, parallel reduction) and implement it using parallel threads
- From hardware's perspective, chip executes simultaneously instructions from **different processes** or **different threads**
- TLP vs ILP
 - Recall concept of "Program Counter" (PC): a register that holds address of next instruction
 - If all instructions executed by the chip in parallel are associated with one PC, we have ILP
 - TLP: the stream of instructions simultaneously processed by the chip associated with more than one PC

Process vs. thread

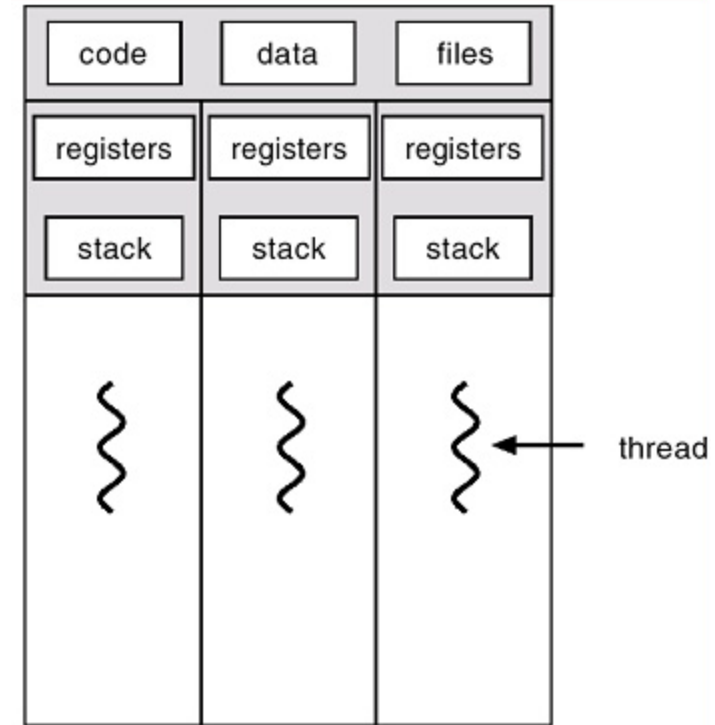
- Thread lives inside a process – the basic execution unit within a process
 - Every process has at least one thread (main thread) entering the main function
- TLP can occur both inside and outside a process:
 - TLP happens when two **processes** such as Matlab and Microsoft Word run on the chip
 - Because two different processes, there are **two different PCs**
 - TLP: happens when a process forks multiple threads to run a function or a parallel algorithm
 - Because of two different threads, there are **two different PCs (one PC per thread)**
- Difference in terms of memory space:
 - Threads (of the same process) run in a **shared memory space**
 - This is how shared-memory parallel computing takes place. Like OpenMP
 - Processes run in **separate memory spaces**
 - This is how distributed-memory parallel computing takes place. Like MPI*

Each thread has its own register file, stack; but not memory space



single-threaded

Physically, it runs on one core
(one CU, one ALU, one register file)

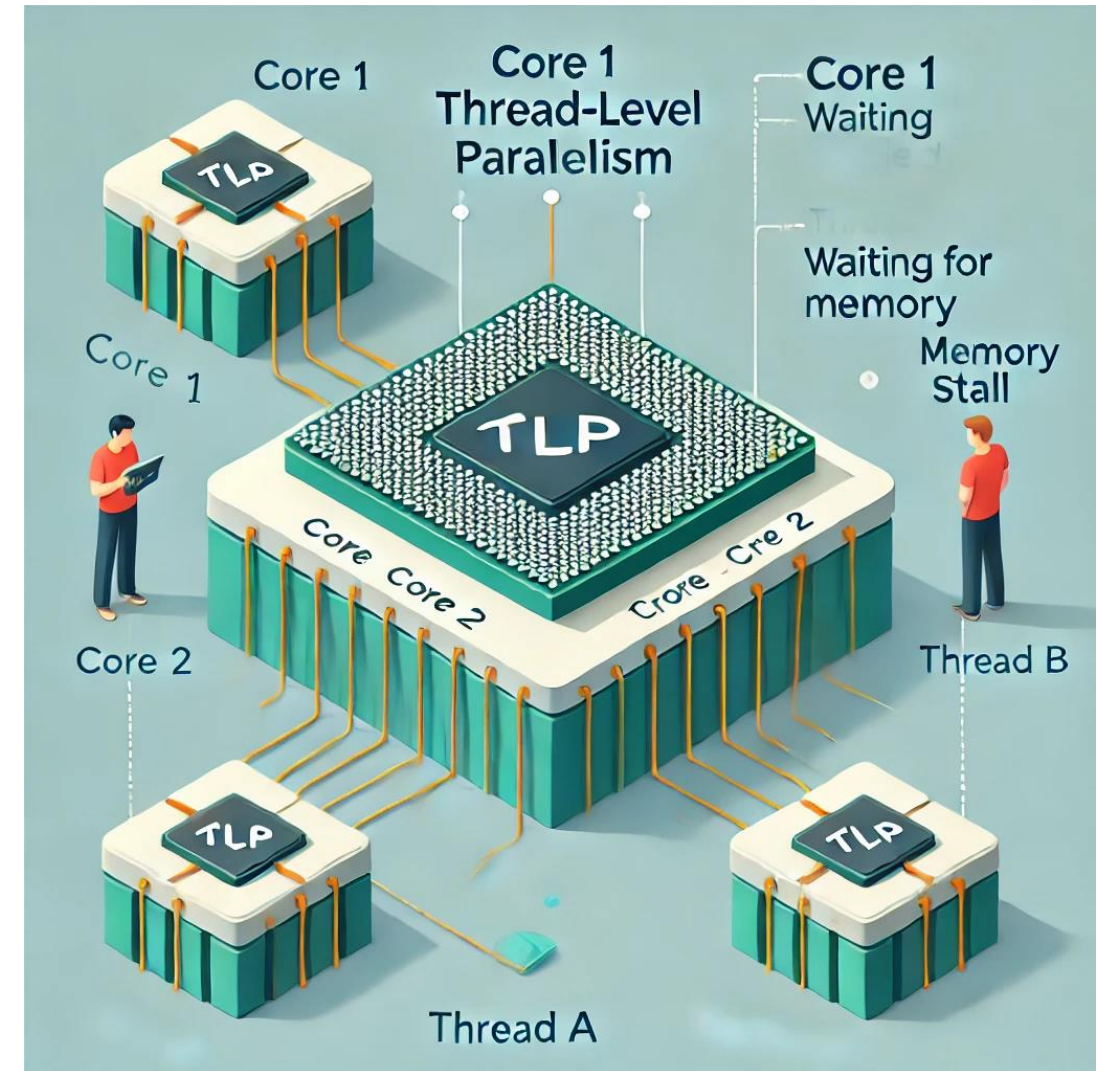


multithreaded

Physically, it runs on one core
(one CU, one ALU, one register file)

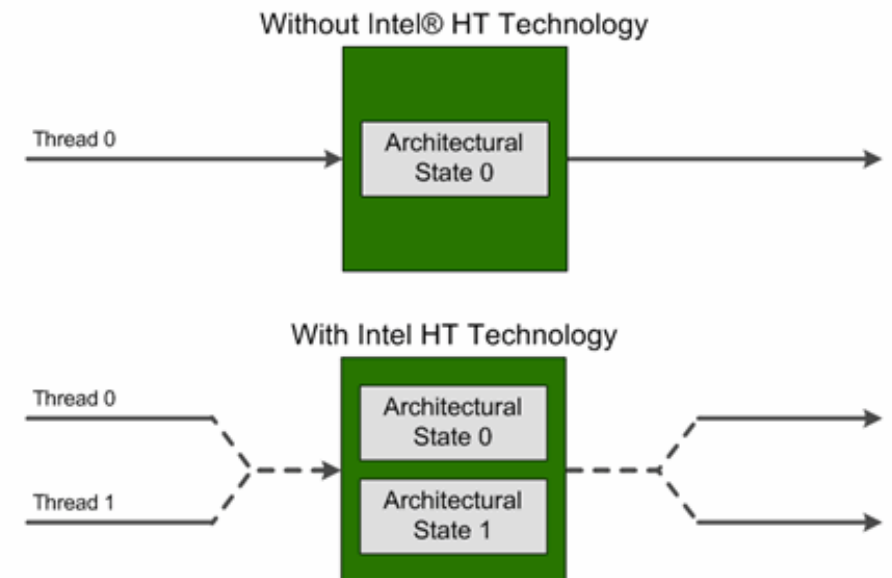
Why did folks come up with this TLP idea?

- Key goal: to improve CPU performance beyond what ILP alone could achieve
- ILP had limits:
 - Can easily hit limits due to data dependencies and branch mispredictions
 - No matter how fancy CPU is, still times when it had to **wait** for previous instructions to finish
- Instead of just making **one** core faster, why not have **multiple cores** running different threads in parallel?
 - If one thread stalls (e.g., waiting for memory), another can execute.
 - Multi-core CPUs can run different threads at the same time
 - Better resource utilization = higher performance!



An example of TLP: Intel's HTT feature

- Typical Intel core supports the “Hyper-Threading Technology” (HTT)
 - HTT allows a single physical CPU core to appear as two logical cores to the operating system, enabling better use of CPU resources by allowing multiple threads to execute simultaneously on each core
- HTT: two threads/processes, each seeing its own stream of machine instructions, active at any given time (“active”: also called “in flight”)
- The scheduler tries to issue instructions from both processes, at the same time
 - The processor core maintains two architectural states, each of which can support its own execution thread
 - Many of the internal microarchitectural hardware resources are shared between the two threads



Intel's HTT Feature

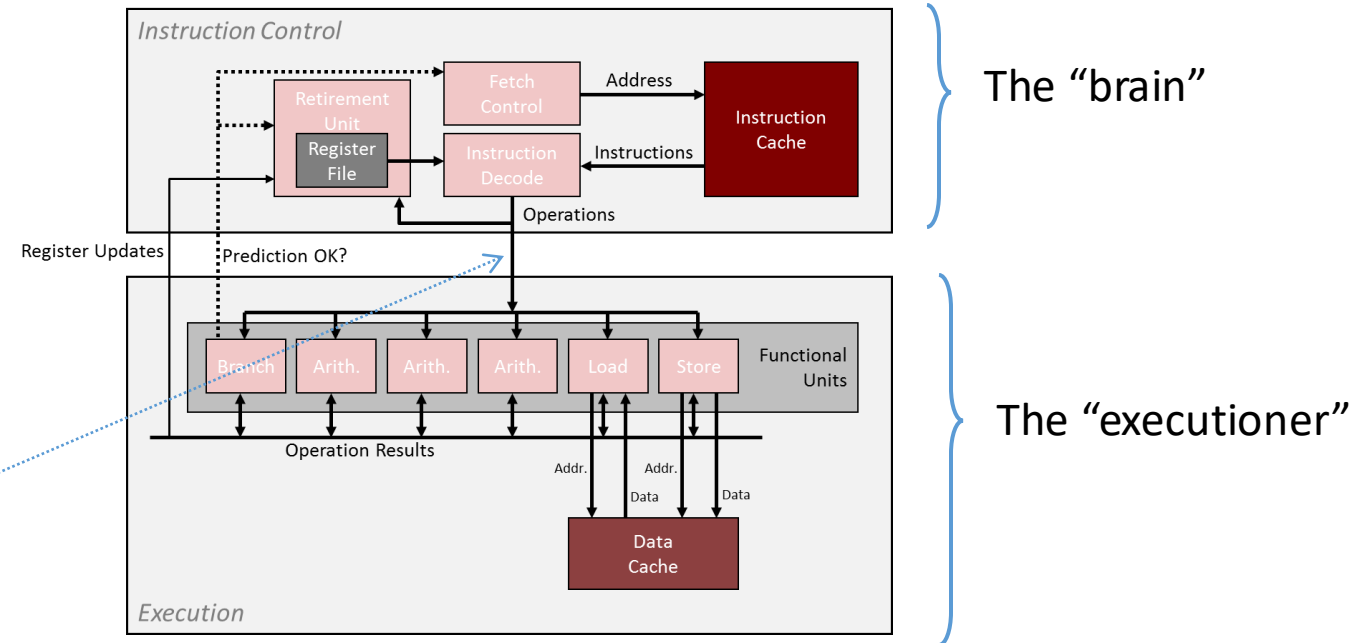
- In general, TLP requires the participation of the Operating System (OS)
 - The OS sees one physical chip as two virtual chips; OS “talks” to both of them
- Is HTT helpful?
 - Yes: when running two modestly demanding processes
 - Example: Running MS-Word and Spotify at the same time
 - Execution threads are independent, when one execution thread stalls the other can be picked up and its PC advanced
 - No: when doing high performance computing, and one stream of instruction saturates the memory bandwidth
 - Nothing left for anybody else to step in and nibble on excess-hardware leftover
- NOTE: as of recently, the trend has been for TLP to be scaled back owing to security issues

TLP: further discussion

- Multi-threading, a possible taxonomy (Hennessey & Paterson)

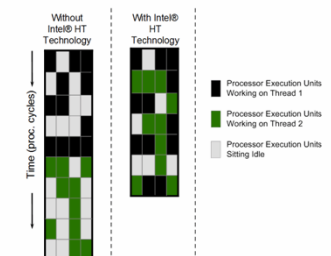
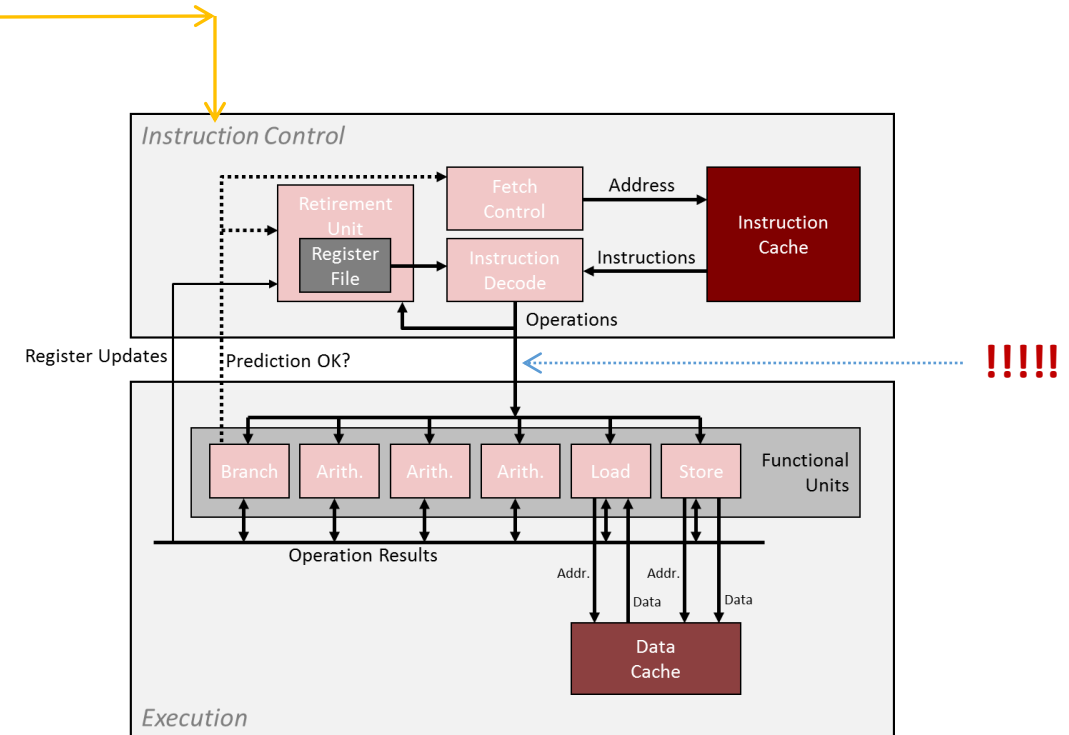
- Coarse grain multi-threading
- Fine grain multi-threading
- Simultaneous multi-threading

- This taxonomy: It all goes back to what goes through this pipe at each clock cycle



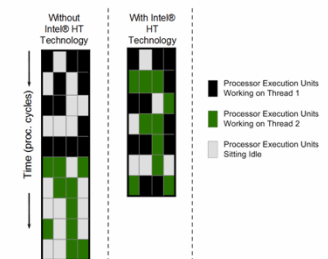
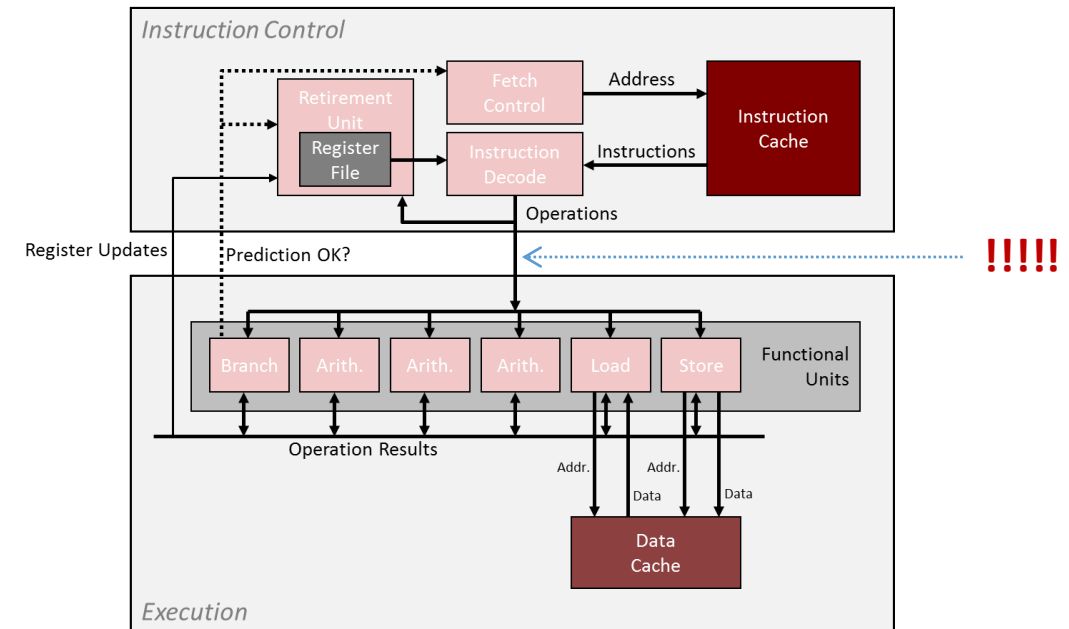
Coarse multi-threading

- The **Instruction Control** issues instructions from one thread for many cycles
 - Bursts of tens or hundreds of instructions
- **Instruction Control** switches to issuing from another thread only when the first thread runs into a long-latency operation
 - Examples: cache miss, interrupt, etc.
 - “long-latency”: Hundreds of cycles or so



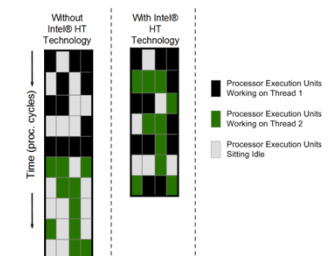
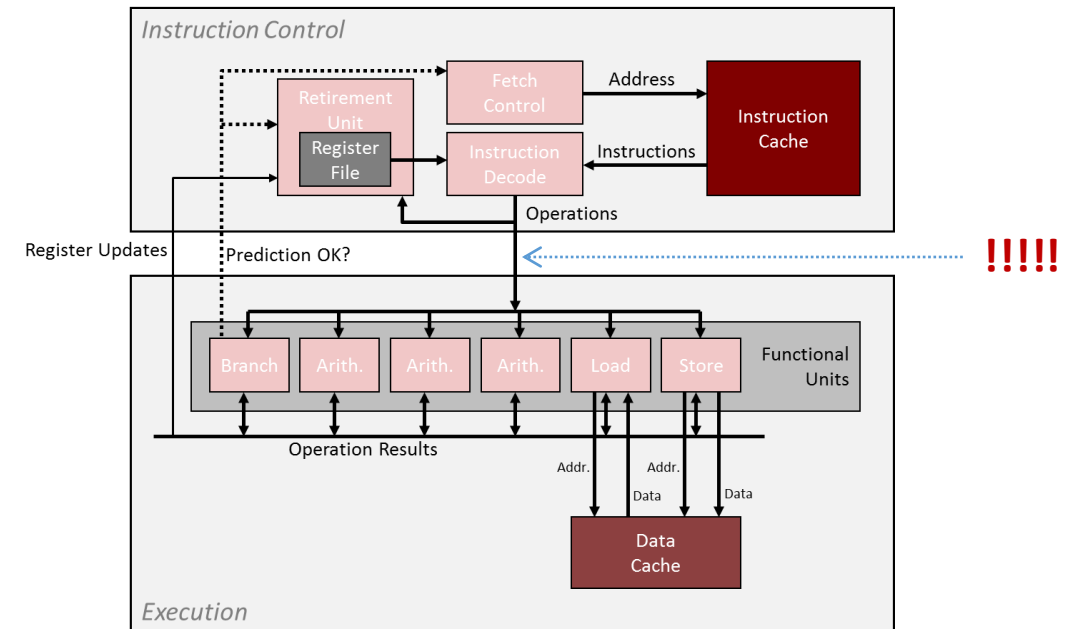
Fine multi-threading

- Instruction Control can issue instructions from a different thread each cycle
 - “Barrel-gun style” processing: thread from where next instruction is picked for execution can change at every cycle
 - GPU computing similar to this
- More nimble/sophisticated compared to coarse multi-threading



Simultaneous multi-threading

- Simultaneous multi-threading is like fine multi-threading with one caveat:
 - Instruction Control can issue instructions from **different** threads in the same cycle
- NOTE:
 - Intel's HTT is a “simultaneous multi-threading” technology



Superscalar processor or TLP processor. Wrapping it up

- Superscalar vs. TLP
 - Superscalar: instructions associated with one PC
 - TLP: instructions associated with two PCs
- Superscalar: one PC on the chip
 - HW has the means to execute more than one instruction per cycle
 - Superscalar: it's more than a “scalar”; i.e., one thread
- TLP: Processor handles instructions from different threads/processes
- **NOTE:** we are not talking about multiple-cores; we are still talking about one CPU core here

Quiz: TLP flavors

- Left-side pic: coarse TLP, fine TLP, or SMT?
 - Not clear from the pic if it's TLP coarse, TLP fine, or even non TLP (might be the operating system engaging in time slicing)
 - To understand if it's TLP or not one needs to know whether the processor is capable of maintaining more than one architectural state in flight
- Right-side pic: coarse TLP, fine TLP, or SMT?

